Universidad de Antioquia
Facultad de Ingeniería
Informática II, 2026-2
Informe preliminar
09 de octubre de 2026

Nombre: Luis Miguel Cohen Teherán; David Santiago Aguirre Mejia

Repositorio: https://github.com/luisCT1/UdeABook-POO-informaticaII

Desafío II: Red social UdeABook

1. Contextualización.

El desarrollo de este segundo desafío nos ha enfrentado a un escenario muy cercano a la ingeniería de software del mundo real, simular la capa lógica de una red social corporativa (UdeABook) enfocada en la gestión eficiente de usuarios, perfiles, álbumes de fotos, mensajería y la red de relaciones interpersonales. A lo largo de esta semana (5 al 9 de octubre) de análisis, nos reunimos en equipo para estudiar a fondo las exigencias del enunciado y evitar improvisaciones a la hora de codificar. Comprendemos que UdeABook no es solo un programa de consola común, sino una plataforma orientada a escalar a futuro, lo que nos exige un rigor absoluto en la elección de las estructuras de datos, el cumplimiento del paradigma de Programación Orientada a Objetos (POO) y la optimización de los recursos de la máquina. El programa debe probarse con al menos 1.000 usuarios y reportar, al finalizar cada funcionalidad, las iteraciones realizadas y la memoria consumida por las estructuras y objetos existentes.

2. Análisis.

Antes de implementar, es necesario definir cómo se representan los datos y qué clase es responsable de cada operación. La información persistente se cargará desde archivos al iniciar el programa. Durante la ejecución, las consultas y modificaciones se harán sobre las estructuras en memoria. Los cambios que deban conservarse se escribirán en los archivos correspondientes; el almacenamiento permanente no se usará como espacio de trabajo temporal.

La eficiencia se evaluará mediante las métricas exigidas, no mediante una afirmación de eficiencia del 100%. Cada algoritmo tendrá un costo asociado a la cantidad de elementos que recorre y a las operaciones que ejecuta. Se priorizará evitar copias innecesarias de objetos, usar referencias o punteros cuando sea apropiado y liberar correctamente la memoria dinámica.

Para lograr todo lo anterior se han identificado los siguientes puntos críticos que deben resolverse antes de escribir la primera línea de código:

- Restricciones técnicas innegociables: el proyecto prohíbe de manera estricta el uso de contenedores de la STL (std::vector, std::list, etc.), estructuras de tipo struct, librerías externas para tareas esenciales y el uso del disco duro como área de procesamiento temporal. Todo debe resolverse con memoria dinámica propia y una arquitectura modular estricta en C++ (.h y .cpp).
- Gestión de la persistencia y sesiones: notamos un reto importante en la lógica de negocio, la cual consiste en una red social real, los usuarios no siempre cierran sesión de forma formal; muchas veces el programa se detiene de golpe. Esperar exclusivamente hasta el cierre de la aplicación para guardar datos habría significado arriesgarnos a perder mensajes, likes o amistades nuevas.
- Eficiencia algorítmica y procesamiento: para cumplir con el requerimiento de eficiencia, debíamos descartar búsquedas lineales lentas o manejo ineficiente de memoria, apostando por estructuras directas en la RAM y operaciones a nivel de bits para las relaciones de amistad.

3. Esquema de tareas y diseño

Para solventar los retos anteriores y cumplir con las expectativas, se estructuró el diseño de la solución bajo las siguientes decisiones técnicas y operativas:

Arquitectura modular y Diagrama de Clases: estructuramos el sistema separando responsabilidades en clases independientes, cada una con su respectivo archivo de cabecera e implementación:

1. RedSocial: controlador principal que administra la lista de usuarios, las credenciales de acceso y centraliza los flujos globales.
2. Usuario: núcleo de información personal (nombre, fechas, ciudad, país, foto de perfil) y propietario de listas dinámicas para mensajes, álbumes, solicitudes y sugerencias.
3. RedAmistades: módulo encargado de la red de conexiones utilizando una matriz de adyacencia optimizada con un arreglo de bits (unsigned long long* bitsDeAmistad), logrando un consumo mínimo de memoria (1 bit por relación) y cálculos ultra rápidos de amigos en común.
4. SolicitudAmistad: representa una solicitud pendiente mediante emisor, receptor y fecha. Separarla de la amistad permite distinguir una solicitud de una relación aceptada.
5. AlbumFotos y Foto: pertenencia para administrar la multimedia del usuario, respetando configuraciones de privacidad (*público, privado, amigos*) y conteo de likes.
6. Mensaje: gestiona el intercambio de textos de hasta 512 caracteres, emisor, receptor, fechas y estados de lectura.
7. MedidorRecursos: acumula las iteraciones atribuibles a cada funcionalidad y estima la memoria ocupada por los objetos y estructuras que existen en ese momento.

8. ListaDinamica<T>: nuestra estructura genérica basada en plantillas (*templates*) y memoria dinámica en el *heap* para reemplazar por completo a los vectores de la STL.
9. FechaHora: representa una fecha y hora y ofrece comparaciones y validación de rangos. Se relaciona con cada clase que almacene una fecha, como Mensaje, SolicitudAmistad y Foto.
10. GestorArchivos: lee los archivos iniciales y persiste los cambios en el formato definido. No participa en las consultas ordinarias una vez cargados los datos.

Estrategia para representar las amistades

Una matriz de adyacencia permite comprobar si dos usuarios son amigos y calcular amigos en común. Para reducir la memoria, cada relación puede representarse con un bit. Si hay N usuarios, una matriz completa requiere N^2 bits, es decir, aproximadamente $N^2/8$ bytes, sin contar el arreglo ni otros metadatos. Para 1.000 usuarios serían cerca de 125.000 bytes en una matriz completa empaquetada. Como la amistad es mutua, se puede estudiar almacenar sólo un triángulo de la matriz, aunque el cálculo de índices añade complejidad. Para el plazo disponible, se implementará primero una representación correcta y verificable; la compactación triangular sólo se adoptará si puede probarse sin comprometer las demás funciones.

Estrategia de Archivos y Persistencia Transaccional

Se propone utilizar archivos de texto con registros separados por líneas y campos delimitados por el carácter |. Como algunos campos podrían contener ese carácter, el formato deberá prohibirlo en dichos campos o definir una regla de escape. Cada lector validará la cantidad de campos y los valores antes de crear los objetos.

Archivo	Contenido propuesto	Actualización
usuarios.txt	nombreUsuario|nacimiento|correo|contrasena|ciudad|pais|inscripcion|rutaPerfil	Solo lectura durante este desafío, según los requisitos actuales.
multimedia.txt	Registros de álbum y fotos asociados al usuario; incluir privacidad, ruta, fecha de captura, tamaño y likes o referencia al archivo de contadores.	Solo lectura si los álbumes y fotos vienen precargados.
amistades.txt	indiceA|indiceB, una sola vez por amistad.	Agregar una línea al aceptar una solicitud.

solicitudes.txt	indiceEmisor|indiceReceptor|fecha	Registrar solicitudes y definir cómo se representan las aceptadas/rechazadas para que no reaparezcan al reiniciar.
mensajes.txt	emisor|receptor|fecha|longitud|estadoLeido|texto	Agregar cada mensaje nuevo.
contadores.txt	Contadores de lectura y likes, con una correspondencia documentada entre posición e índice de usuario/foto.	Actualizar de forma controlada al cambiar un estado o like.

La propuesta inicial de usar campos de ancho fijo para contadores puede funcionar, pero exige documentar exactamente el orden de los registros, el tamaño de cada campo y cómo se localiza cada valor. Si se sobrescribe un valor en su posición, debe comprobarse que la escritura no cambie la longitud del archivo. Para reducir riesgos, se probará primero la lectura y escritura con archivos pequeños y luego con el conjunto de prueba.

Algoritmos principales previstos

1. Inicio de sesión: buscar el nombre de usuario y validar la contraseña; si las credenciales coinciden, conservar una referencia al usuario activo y mostrar el menú.
2. Aceptar solicitud: verificar que la solicitud esté pendiente, registrar la amistad en ambos sentidos lógicos, retirar o marcar la solicitud como atendida y persistir el cambio.
3. Sugerencias de amistad: recorrer los amigos del usuario y los amigos de estos, excluir al usuario actual, sus amigos y las solicitudes incompatibles; contar coincidencias y seleccionar las K mejores.
4. Mejores amigos: calcular las coincidencias y seleccionar tres perfiles distintos aplicando los pesos de prioridad definidos en el enunciado. Evitar repetir usuarios en una misma selección.
5. Carrusel de fotos: comprobar la privacidad del álbum según la relación con su propietario y recorrer la lista circularmente al avanzar o retroceder.
6. Mensajes no leídos: recorrer los mensajes recibidos, filtrar los no leídos, ordenarlos por fecha de envío si la estructura no garantiza ese orden, mostrarlos y marcar los presentados como leídos.
7. Búsqueda por fechas: recorrer los mensajes recibidos y seleccionar los que estén dentro del intervalo inclusivo indicado.
8. Medición: reiniciar el contador de iteraciones antes de cada funcionalidad, registrar operaciones relevantes y calcular una estimación documentada de la memoria ocupada.

Medición de recursos y pruebas

El contador de iteraciones se incrementará en los ciclos y operaciones relevantes, incluidos los recorridos realizados por métodos llamados internamente. Se documentará qué operaciones cuentan como iteración para que las mediciones sean comparables. La memoria se estimará sumando el tamaño de los objetos, arreglos dinámicos y cadenas reservadas, incluidos los datos dinámicos a los que apuntan. Se evitará contar dos veces una misma asignación compartida. La estimación debe incluir variables locales y parámetros por valor implicados en la funcionalidad, tal como pide el enunciado.

El programa auxiliar para generar datos de prueba se mantendrá separado de la entrega principal. El conjunto de prueba tendrá al menos 1.000 usuarios, amistades, solicitudes, mensajes distribuidos en varias semanas y usuarios con dos álbumes y cinco fotos por álbum para demostrar los recorridos. Se comprobarán casos límite: credenciales incorrectas, solicitud duplicada, amistad existente, mensaje vacío, texto superior a 512 caracteres, rango de fechas inválido, privacidad del álbum y likes repetidos.

Organización modular y plan de trabajo

Cada clase tendrá archivos de cabecera e implementación (.h y .cpp), y se definirá una responsabilidad clara para evitar dependencias circulares. La implementación se abordará en este orden: ListaDinamica y FechaHora; Usuario, Foto, AlbumFotos y Mensaje; RedAmistades y SolicitudAmistad; GestorArchivos; RedSocial; MedidorRecursos; integración del menú y pruebas con 1.000 usuarios. Se harán commits frecuentes en el repositorio y se validará cada módulo antes de integrar el siguiente.

Consideraciones pendientes

- Definir si las solicitudes se conservan como historial o si se eliminan/marcan al aceptarlas o rechazarlas. El formato actual debe permitir distinguir solicitudes pendientes de las atendidas.
- Definir un identificador estable para cada foto, de modo que los likes se vinculen siempre a la foto correcta.
- Definir la correspondencia entre nombres de usuario e índices usados por las estructuras de amistades, y asegurar que los índices no cambien durante la ejecución.
- Definir cómo se resuelven errores de lectura, campos incompletos, archivos ausentes y fallos de escritura.
- Evitar afirmar que la persistencia es transaccional si no existe un mecanismo para recuperar una escritura parcial. En esta versión se describirá el guardado inmediato de los cambios.

- Comprobar que el modelo cubra todas las funcionalidades del enunciado y no implementar funciones que no se solicitan, dado el tiempo disponible.

La solución propuesta separa la lógica de negocio, las estructuras de datos, la persistencia y la medición de recursos. La representación compacta de amistades se conserva como una alternativa prioritaria, pero la implementación se organizará por etapas para asegurar primero el funcionamiento correcto de las tareas requeridas. El diseño podrá ajustarse durante la codificación, documentando los cambios y verificando que se mantengan las restricciones del desafío.